Sensiq: An MCU-Based IoT Architecture for Environmental Monitoring

Technical Reports: CL-2026-42, **selfref** *2026*

Department of Electrical Engineering, Media, and Computer Science
Ostbayerische Technische Hochschule Amberg-Weiden
Amberg, Germany

***Abstract*—Background: Reliable environmental monitoring is essential for laboratory and facility safety, yet many systems trade off scalability, security, or real-time access. Objective: This concept paper proposes Sensiq, an MCU-based IoT architecture intended to enable secure, low-latency live monitoring and scalable historical storage for environmental sensor data.**

Methods: We describe a serverless AWS-based pipeline in which ESP32 edge devices publish JSON-encoded measurements (temperature, humidity, gas, flame) over MQTT/TLS. The architecture specifies Lambda-based validation and persistence to DynamoDB and S3, and REST endpoints for 'GET /live' and 'GET /history'. We propose a configurable simulator to exercise ingestion and retrieval without physical hardware and to guide future implementation and testing.

Planned evaluation: This paper outlines an evaluation plan using the simulator to validate end-to-end ingestion, API latency, data integrity, and storage correctness once the MVP is implemented.

Conclusion: Sensiq defines a lightweight, extensible foundation for laboratory environmental monitoring. Future work focuses on implementing the MVP, performing the described evaluations, adding a web dashboard and notification services for visualization and alerts.

***Index Terms*—IoT, Environmental Monitoring, ESP32, MCU, MQTT, Serverless Architecture, AWS**

I. Introduction

A. Motivation

In modern laboratory and production environments, maintaining stable ambient conditions is vital for experimental validity and product quality. Conventional monitoring systems often lack real-time accessibility, scalability, or automated alerting capabilities. This project addresses these gaps by integrating cost-effective ESP32 edge hardware with a robust AWS cloud architecture. By transitioning from isolated local measurements to a centralized, serverless IoT platform, we enable seamless data persistence, interactive visualization, and immediate incident response. The objective is to provide lab managers and technicians with a scalable, high-availability tool that ensures operational safety and data-driven quality assurance.

?? gives an overview of related work and technologies that inspired and influenced the design of our system. The core requirements are defined in Section II with a focus on user-centered design, including detailed user stories and acceptance criteria for the minimum viable product (MVP). The technical concept and architecture of the system are described in Section III. Section IV summarizes the key findings and outlines future work and next steps for the project.

B. Key Technical Components

The proposed system is built around an ESP32 edge device connected to multiple environmental sensors, such as temperature, humidity, gas, and flame sensors. Sensor readings are pre-processed locally and transmitted via encrypted MQTT/TLS to protect data confidentiality and ensure device authentication during communication.

AWS IoT Core serves as the secure entry point between the devices and the cloud backend. It receives incoming measurements and makes them available for further processing of other AWS services. Live sensor values are handled by a dedicated processing pipeline and exposed through a REST interface for external access, while historical measurements are stored and made available through a separate REST endpoint for time-series retrieval and analysis.

Future extensions may include a frontend dashboard that provides user-oriented visualization and analysis tools for monitoring trends and comparing sensor values over time. In addition, AWS SNS can be integrated later to send email notifications to users when critical thresholds are exceeded.

II. Requirements

The requirements for the proposed system are defined in the form of persona-based user stories including acceptance criteria and prioritization for the minimum viable product (MVP).

A. Project Personas

To ensure the system meets the diverse needs of all stakeholders, the requirements are guided by the following personas. Each persona has a specific role, defined goals, and distinct interests in the system:

Chemist Christian Focuses on exact environmental conditions. His primary goal is to monitor precise laboratory temperatures to accurately run experiments and evaluate environment-dependent reactions.

Occupational Safety Officer Olivia Prioritizes employee well-being. She requires immediate, clear indicators of air quality safety to proactively enforce protective measures and prevent workplace health hazards.

Facility Manager Martin Responsible for building operations and rapid incident response. He needs instant, unmissable alerts regarding critical sensor thresholds to take immediate corrective actions before physical risks escalate.

Project Manager Pia Looks at the big picture and long-term optimization. She aims to analyze historical data trends to identify recurring patterns, correlate them with facility operations, and implement targeted strategies for improving environmental conditions.

B. User Stories

1) Real-time Temperature Monitoring:

As Chemist Christian, I want to see the exact current temperature in the laboratory, so that I can accurately plan my experiments and evaluate temperature-dependent reaction rates.

Acceptance Criteria:

- The dashboard displays the current temperature precisely in degrees Celsius. Input: latest DB temperature readings. Output: exact numerical temperature value.
- The temperature value updates automatically when a new reading arrives — no manual reload required. Test: write new temperature values to DB; display must update within 10 s.
- Displayed temperature values are at most 30 s old. Test: compare value timestamps with system time; difference $\leq$ 30 s.
- If no current temperature data is available (gap $>$ 30 s), a notice about missing data is shown instead.

2) Workplace Air Quality Status:

As Occupational Safety Officer Oliva, I want to see at a glance whether the air quality in the workspace is currently safe or critical, so that I can immediately initiate protective measures for the employees if necessary.

Acceptance Criteria:

- The dashboard shows the air quality as a colour-coded indicator based on safety thresholds: Green (safe) and Red (critical). Input: latest DB air quality readings. Output: correct status colour.
- The status colour updates automatically when a new reading arrives — no manual reload required. Test: write new air quality values to DB; display must update within 5 s.
- The displayed status is based on data that is at most 30 s old. Test: compare value timestamps with system time; difference $\leq$ 30 s.

3) Critical Alert:

As a facility manager Martin, I want an immediate, unmissable notification the second a sensor reading hits a critical limit, empowering me to take instant corrective action before the situation escalates into a physical health risk.

Acceptance Criteria:

- An email notification is sent automatically within 10 s of a threshold exceedance.
- The notification contains: measured value (incl. unit), measurement timestamp, and alert message type.
- Consecutive exceedances of the same type within 5 min do not trigger a further email (debounce).
- The notification is delivered even if the frontend is not open at trigger time.

4) 24-Hour History Chart:

As Project Manager Pia, I want to analyze the sensor data from the past 24 hours in a clear historical chart, so that I can identify recurring patterns of poor air quality, correlate them with specific facility operations, and implement targeted ventilation strategies.

Acceptance Criteria:

- Dashboard displays a line chart with readings from the last 24 hours.
- Chart is fully interactive within 3 s of page load.
- Periods without data are shown as visible line breaks — no silent interpolation.
- Chart updates automatically when new readings arrive, without a page reload.

5) Custom History View:

As Data Scientist Daniela, I want to extract historical sensor readings with a customizable time range and sampling resolution, so that I can generate precise, high-granularity datasets to train our predictive air-quality models.

Acceptance Criteria:

- Dashboard provides date pickers for start/end and a resolution selector (hour/day).
- A query for 30 days at daily resolution returns results in under 5 s.
- If an end date before the start date is entered, a clear error message is shown and no query is executed.
- Periods without data appear as an explicit empty row in the table and as a line break in the chart.

6) Sensor Status:

As IT Administrator Ian, I want to instantly verify the connection status and the last heartbeat timestamp of each sensor, so that I can proactively detect offline devices and troubleshoot network dropouts before data gaps occur.

Acceptance Criteria:

- Sensor status Active/Inactive is prominently visible. Status switches to Inactive when the last data point is more than 2 min ago.
- Date and time of the last received reading are displayed next to the status (format: `DD.MM.YYYY HH:MM:SS`).
- Status indicator updates automatically — no manual reload required.

- If the database connection is temporarily unavailable, a notice is shown instead of the status.

C. Definition of Done

A user story is considered complete only when, in addition to its specific acceptance criteria, all of the following quality standards are met:

- Code Review: Reviewed and approved by at least one other developer (four-eyes principle).
- Tests: Unit tests written and passing; coverage $\geq 40\,\%$.
- Documentation: Technical docs and API interfaces updated where applicable.
- Deployment: Feature successfully deployed.
- Functional Test: Feature manually tested in staging without issues.

D. Prioritisation (MoSCoW)

The user stories are prioritized using the MoSCoW method, which categorizes requirements into Must Have, Should Have, Could Have, and Won't Have, as shown in Table I.

Table I
MoSCoW Prioritization of User Stories

Category	User Stories
Must Have	Current Status Display, Critical Alert, Workplace Air Quality
Should Have	Sensor Status
Could Have	24-Hour History Chart
Won't Have	Custom History View

E. Minimum Viable Product (MVP)

The MVP implements a server-side IoT pipeline on Amazon Web Services (AWS). It focuses on ingesting simulated sensor data via MQTT, processing and storing it within the AWS ecosystem, and providing access through a Web API. This serves as the technical foundation for the system.

Instead of physical hardware, a simple script simulates realistic sensor data. Messages are transmitted via the MQTT protocol to AWS IoT Core, ensuring a lightweight and software-defined testing environment.

A REST API provides two core endpoints: `GET /live` returns the most recent measurement, while `GET /history` retrieves chronological data from a selected time range.

To focus on core infrastructure, the following elements are excluded from this MVP but are planned for future iterations:

- Graphical frontend dashboard for data visualization.
- Physical ESP32 hardware and real sensors.
- Automated user notifications (e.g., Push or Email alerts).

III. Technical Concept

This Section describes the overall technical architecture of the proposed system, including the embedded hardware, communication protocols, cloud infrastructure, data model, and planned extensions. The focus lies on the interaction of IoT devices with a scalable, serverless cloud backend for real-time and historical data processing.

```json
{
  "deviceId": "esp32-lab-01",
  "runningSince": "10000",
  "location": "Lab A",
  "measurements": {
    "dht11_temperature": {
      "value": 20,
      "unit": "degree C"
    },
    "dht11_humidity": {
      "value": 50,
      "unit": "%"
    },
    "ky026_flame": {
      "value": 4000,
      "unit": "analog"
    },
    "ky028_termistor": {
      "value": 22,
      "unit": "degree C"
    }
  }
}
```

Listing 1. Example of the JSON data model for sensor value encapsulation on the ESP32.

A. Data Acquisition and Transmission

The local ESP32 microcontroller is equipped with multiple sensors for monitoring environmental conditions.

The DHT11 sensor measures temperature and humidity with a decent accuracy for indoor applications. This is complemented by the KY-028 thermistor for high precision temperature readings. The KY-026 flame sensor enables the detection of uncontrolled fire sources by measuring the intensity of infrared light emitted by flames [**ky026flame**].

An additional gas sensor (e.g., MQ135) must be determined based on the specific gases of interest, such as volatile organic compounds (VOCs), carbon monoxide, or others relevant to the use case.

The sensor data is read periodically, pre-processed on the ESP32, and encapsulated into a structured data object for transmission, as illustrated in Listing 1. Each message contains device metadata (ID, location, uptime) and a nested structure for sensor measurements for further extensibility.

Messages are published to AWS IoT Core with the MQTT protocol and secured using TLS [**aws**]. Certificate-based authentication ensures device integrity and prevents unauthorized publishing to the cloud backend.

B. Technology Stack

The following programming languages and technologies are utilized across the different components of the system:

- C++: ESP32 firmware for sensor data acquisition and MQTT communication.

- Python: Implementation of AWS Lambda functions for data processing and validation.
- TypeScript: Infrastructure definition and backend logic within AWS services.
- React: Web-based frontend for visualization and interactive data analysis.

C. Cloud Processing Architecture

Incoming MQTT messages are processed within AWS using a serverless, event-driven architecture. The cloud backend is logically divided into two branches for data handling:

Live data is made available through a RESTful API endpoint `GET /live`, optimized for low-latency access to the most recent sensor values [**restful2000**]. Historical data is persisted for long-term storage and analysis, accessible via `GET /history` for in-depth time-series evaluation and trend analysis.

Additionally, an alerting mechanism is planned to be implemented using AWS SNS to notify users via email when certain thresholds are exceeded, such as high temperature. Future extensions may include the development of a React-based frontend dashboard for visualizing historical trends for specific time periods, enabling time-based filtering and comparison of environmental conditions.

The overall architecture is represented in Figure 1, illustrating the flow of data from the ESP32 device through AWS to the API endpoints and potential alerting services.

D. Testability

The system's testability varies across services. Lambda functions are fully unit-testable via SAM CLI and support integration testing using DynamoDB Local or mocked dependencies. DynamoDB and S3 can be tested locally with DynamoDB Local and LocalStack respectively, while API Gateway supports end-to-end testing through SAM's local API emulation. Athena and Glue lack free local equivalents, requiring LocalStack Pro for integration tests. AWS IoT Core cannot be tested locally at all, with Device Advisor being the only available validation tool. SNS supports partial local mocking but can be fully integration-tested via LocalStack.

E. Cost Estimation

The calculation of Table II is made to estimate the average operating cost per month for one device. Assumptions of 3 unique users, 8 hours per day and 22 days per month are made. As depicted, the difference between Free Tier and Post Free Tier costs is marginal.

IV. Conclusion and Future Work

Sensiq presents a practical IoT architecture for environmental monitoring in laboratory and facility settings. Its core contribution is a serverless AWS-based pipeline that ingests sensor data from ESP32 devices via MQTT/TLS, validates the measurements, and stores them for both immediate and historical access.

The paper defines the system requirements through persona-based user stories and prioritizes an MVP that centers on live status display, air-quality assessment, sensor alerts, and time-series access through a Web API. This scope establishes a clear path from edge sensing to cloud processing and later visualization.

Future work will implement and evaluate the MVP, including data ingestion, API behavior, and storage correctness. Subsequent extensions may add a graphical dashboard, notification services, and richer historical analysis to support operational monitoring and decision-making.

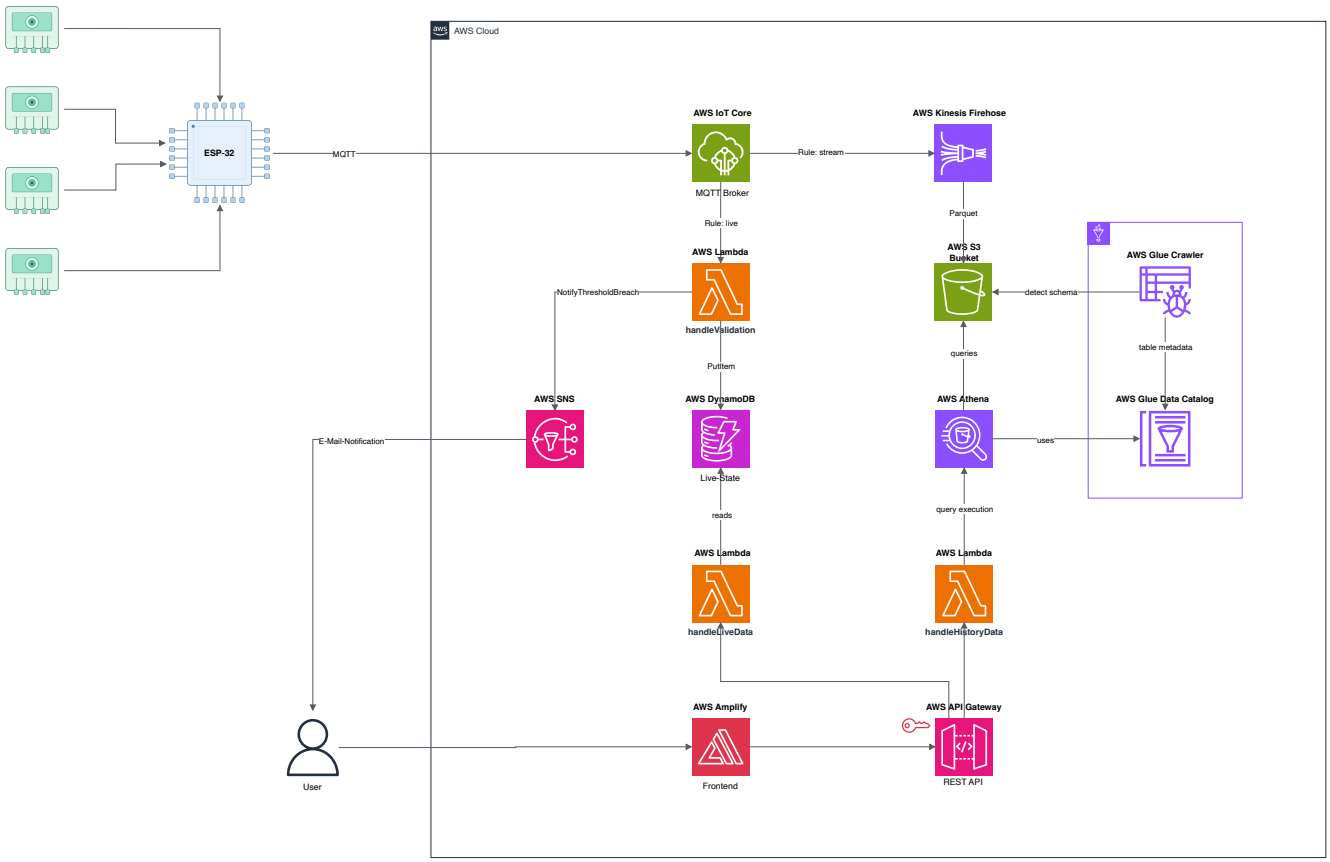

Figure 1. Cloud Processing Architecture

Table II
COST ESTIMATION FOR AWS SERVICES

Service Details	**Usage/Quantity**	**Free Tier**	**Post Free Tier**
IoT Core [**awsIotCore**]	518k msgs, 268k billed	∼$0.27	∼$0.27
Lambda – handleValidation [**aws-lambda**]	518k inv, 128 MB / 200 ms	$0.00	∼$0.06
Lambda – handleLiveData [**aws-lambda**]	228k inv, 128 MB / 200 ms	$0.00	∼$0.04
Lambda – handleHistoryData [**aws-lambda**]	660 inv, 128 MB / 200 ms	$0.00	∼$0.00
DynamoDB [**aws-dynamodb**]	518k WCU, 228k RCU, storage	∼$0.01	∼$0.08
S3 (Parquet) [**aws-s3**]	∼0.5 GB stored	∼$0.01	∼$0.02
Athena [**aws-athena**]	660 queries × 50 MB = 33 GB scanned	∼$0.16	∼$0.16
API Gateway [**aws-api-gateway**]	229k requests	$0.00	∼$0.08
SNS [**aws-sns**]	Minimal alert volume	$0.00	∼$0.01
Glue Crawler [**aws-glue**]	One-time run	∼$0.00	∼$0.00
Total		**∼$0.46**	**∼$0.73**